DEFLOW: DEEP EQUILIBRIUM FLOW MAXIMUM-ENTROPY REINFORCEMENT LEARNING

Anonymous authors
Paper under double-blind review

ABSTRACT

Expressive flow policies can represent multiple high-value actions, but sampling them with few integration steps while retaining tractable action densities remains challenging. We introduce DEFlow, a deep equilibrium flow policy that replaces explicit Euler sampling with implicit Euler integration and solves all intermediate actions jointly as an equilibrium system. We combine implicit differentiation with a discrete change-of-variables formula for the final-action density, enabling off-policy training within soft actor–critic without backpropagating through solver iterations. DEFlow achieves an average relative improvement of approximately 10% in mean successful coverage over the strongest baseline across four Multi-Goal docking variants. It also achieves the highest mean time-averaged return on four of five manipulation tasks, while remaining broadly competitive across five MuJoCo tasks. These results support implicit flow policies for tasks requiring both diversity and precision, at the cost of additional nonlinear and linear solves.[1]

1 INTRODUCTION

Soft actor–critic (SAC) is a strong baseline for continuous control. It combines data-efficient off-policy learning with entropy-driven exploration (Haarnoja et al., 2018a;b). Its standard actor, however, is a single tanh-squashed diagonal Gaussian. This distribution is easy to sample from and has a closed-form density, but it cannot easily represent several distinct high-value actions at the same state (Haarnoja et al., 2017). Diffusion and flow-matching policies offer a more expressive alternative. They generate actions by transforming simple noise through multiple learned steps and can represent complex, multimodal distributions (Wang et al., 2023; Lipman et al., 2023). The challenge is to gain this expressivity while keeping SAC training practical. Sampling and actor differentiation now involve a multi-step rollout, and the final-action log-density required by the entropy objective may be hard to obtain.

Existing off-policy methods resolve this tension by design compromise. Some give up the maximum-entropy objective altogether: Flow Q-Learning distills a multi-step generator into a one-step actor (Park et al., 2025), consistency policies replace the long sampler with one- or few-step generation (Ding & Jin, 2024), and Diffusion-QL combines diffusion behavior cloning with value maximization without evaluating the policy likelihood (Wang et al., 2023). Q-score matching (QSM) (Psenka et al., 2024) directly matches the policy score to the critic's action-gradient and avoids backpropagating through the complete sampler. SAC-Flow (Zhang et al., 2025), the direct parent of this work, treats its explicit K-step flow rollout as a sequential model and stabilizes the recurrence with a gated velocity field. DIME retains a multi-step denoising policy and trains its transitions through approximate inference toward a Q-defined energy-based target (Celik et al., 2025). These methods make expressive generative actors trainable off-policy, but leave a separate numerical question: how should a multi-step policy be sampled when its learned velocity field becomes difficult to integrate with a small number of explicit steps? We investigate implicit-Euler sampling as an alternative.

We introduce DEFlow, a flow policy built around implicit-Euler sampling. We retain SAC-Flow's gated velocity field and multi-step actor, but evaluate each velocity at the unknown next action, replacing the explicit Euler recurrence with implicit-Euler equations (Hairer & Wanner, 1996). This integration rule is well suited to stiff dynamics, making it a natural alternative when a

[1] Code available at https://anonymous.4open.science/r/deflow-81BE/README.md.

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

learned field is poorly resolved by a few explicit steps. To implement the sampler, we collect all T intermediate actions into one small deep-equilibrium system and solve for them jointly (Bai et al., 2019; Walker & Ni, 2011). The forward pass uses a root solver to obtain the trajectory, while the backward pass is formulated through implicit differentiation rather than backpropagation through the solver iterations. Under the stated regularity assumptions, an exact equilibrium also admits a final-action log-density through discrete change of variables. The implementation evaluates the density and implicit gradient along the numerically obtained trajectory.

In summary, our contributions are threefold:

- **A joint equilibrium formulation of flow policies.** We formulate the full implicit-Euler action trajectory as a single deep equilibrium system, providing an alternative to explicit rollouts for expressive policies.
- **Maximum-entropy policy learning.** We train DEFlow within SAC using implicit gradients and a final-action density formula, exact at equilibrium under the stated assumptions.
- **Comprehensive empirical evaluation.** We study DEFlow across precision docking, manipulation, and MuJoCo benchmarks, with matched explicit-Euler baselines, step-count ablations, and numerical density verification. The experiments characterize both the strengths and the limitations of implicit flow sampling.

2 Background and Related Work

2.1 Maximum-Entropy Reinforcement Learning

We consider an infinite-horizon Markov decision process with state $s \in \mathcal{S}$, action $a \in \mathcal{A} \subseteq \mathbb{R}^{d_a}$, transition kernel $p(s' \mid s, a)$, reward $r(s, a)$, and discount $\gamma \in (0, 1)$, where d_a is the action dimension. Soft actor–critic (SAC) trains a policy π_θ off-policy on transitions drawn from a replay buffer $\mathcal{D}$ to maximize the maximum-entropy return $J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\big[\sum_t \gamma^t \big(r(s_t, a_t) + \alpha\, \mathcal{H}(\pi_\theta(\cdot \mid s_t))\big)\big]$, where the temperature $\alpha > 0$ trades reward against entropy (Haarnoja et al., 2018a;b). With twin critics Q_{ϕ_1}, Q_{ϕ_2}, the actor balances expected value against the log-density of its sampled actions. The same log-density enters the soft target for the critics. Section 4.2 gives the actor and critic objectives used by DEFlow.

SAC provides the actor–critic framework used in this work. Its standard actor parameterizes a state-conditioned Gaussian distribution whose samples are squashed by $\tanh$ to satisfy the action bounds (Haarnoja et al., 2018b). The underlying Gaussian is unimodal, which limits the policy's ability to capture complex, multimodal action distributions.

2.2 Flow and Diffusion Policies

A flow policy is the expressive actor we adopt. Rather than a single Gaussian draw, it turns base noise $z \sim p_0$ into an action by transporting it through a learned velocity field. The state-conditioned transport follows $\dot{a}(t) = v_\theta(a(t), s, t)$, starting from $a(0) = z$ for $t \in [0, 1]$. A numerical integrator turns this motion into T discrete steps; the endpoint is then squashed to keep the action bounded. Flow matching (Lipman et al., 2023) provides a way to learn such fields from data. By transporting samples from a simple base distribution, a flow can represent complex action distributions with multiple distinct modes.

Representing a richer action distribution, however, does not by itself make the policy easy to optimize with SAC. The standard actor update propagates gradients from the critic through the sampled action to the policy parameters. Flow policies admit this differentiation, but direct optimization requires propagating the learning signal through the entire sequence of sampling steps. This can be computationally expensive and can lead to vanishing or exploding gradients, making stable policy improvement a separate challenge from learning to reproduce an action distribution (Park et al., 2025; Zhang et al., 2025).

Our main comparisons concern expressive policies learned through online interaction. SAC-Flow (Zhang et al., 2025), our direct predecessor, combines a gated velocity field with explicit Euler integration and a surrogate entropy objective within SAC. DIME (Celik et al., 2025) instead uses a

variational maximum-entropy bound, while Q-score matching (QSM) (Psenka et al., 2024) learns the policy score from the critic's action-gradient without evaluating a density.

2.3 Implicit Models and Differentiation

Deep equilibrium (DEQ) models are implicit-layer networks (Bai et al., 2019): rather than stack many explicit layers, a DEQ defines its output as the fixed point $Z^\star = g_\theta(Z^\star)$ of a single weight-tied map and computes it with a root-finder. When g_θ is a contraction, the iteration $Z^{k+1} = g_\theta(Z^k)$ converges at a linear rate bounded by its Lipschitz constant. Anderson acceleration (Walker & Ni, 2011) uses a history of iterates and residuals to accelerate this solve.

The gradient follows from the solved equilibrium, without differentiating through the solver iterations. For a continuously differentiable map with nonsingular $I - \partial_Z g_\theta(Z^\star)$, implicit differentiation turns the nonlinear forward problem into a linear system in the backward pass. Standard DEQ implementations solve the adjoint system iteratively. Jacobian regularization can improve the convergence of these solves (Bai et al., 2021). A complementary perspective treats backpropagation as block-triangular back substitution (Edelman et al., 2024). We use the equilibrium formulation and implicit differentiation to construct DEFlow in Section 4.

3 The Finite-Step Sampling Problem

At a fixed state, the maximum-entropy optimum (Haarnoja et al., 2017; 2018b) is

$$\pi^\star(a \mid s) \propto \exp(Q^\star(s,a)/\alpha)\,. \tag{1}$$

It can therefore contain several narrow modes corresponding to different valid actions. The tanh-Gaussian actor used by SAC is unimodal before squashing, whereas a flow policy can represent these modes if its finite-step sampler preserves them.

A finite inference budget. At a fixed state, explicit and implicit Euler use

$$a_{k+1}^{\exp} = a_k + \Delta t\, v_\theta(a_k, s, t_k), \tag{2}$$

$$a_{k+1}^{\mathrm{imp}} = a_k + \Delta t\, v_\theta(a_{k+1}, s, t_{k+1}). \tag{3}$$

where $t_k = k\Delta t$ and $\Delta t = 1/T$. Real-time control limits latency and field evaluations, so the usable T may be small and Δt correspondingly large. Solving an implicit step may require multiple field evaluations, increasing its computational cost.

Three distinct numerical effects. Coarse discretization affects (i) the endpoint, (ii) the step map, and (iii) the policy gradient. For the one-step maps $F_k\colon a_k \mapsto a_{k+1}$ and $J_k = D_a v_\theta(a_k, s, t_k)$, the step Jacobians are

$$D_a F_k^{\exp} = I + \Delta t\, J_k, \qquad D_a F_k^{\mathrm{imp}} = (I - \Delta t\, J_{k+1})^{-1}\,. \tag{4}$$

so the two schemes can have different conditioning on the same field. In the fixed-state mixture analysis of Appendix A.1, the exact field is smooth, but its largest derivative grows at least like $m^2\sigma^{-3}$ as the mode width σ decreases. Writing $\pi^\star_{\Delta t}$ for the law induced by T explicit Euler steps of the exact field and $\pi^\star$ for the two-mode surrogate target of that appendix, explicit Euler incurs

$$\Delta t \geq 2\sqrt{2}\,\sigma \quad \Longrightarrow \quad W_2(\pi^\star_{\Delta t}, \pi^\star) \geq \frac{\sigma}{2\sqrt{2}}, \tag{5}$$

and at $T = 2$ its error tends to a positive limit as $\sigma \to 0$ (Eq. (29)). The same analysis bounds the discrete gradient at no more than a fraction $3(\sigma/\Delta t)^2$ of the exact one for a specified Gaussian subfamily and terminal loss, so it nearly vanishes in the same coarse-grid regime. Coarse explicit integration can introduce errors in both the sampled distribution and its gradients even when every step remains locally invertible.

A coarse-step empirical illustration. Figure 1 shows that implicit Euler achieves more accurate robot reaching than explicit Euler at low step counts for the tested fixed contracting flow. With just two steps, the joint DEQ sampler concentrates endpoints near the target and achieves 97.8% success,

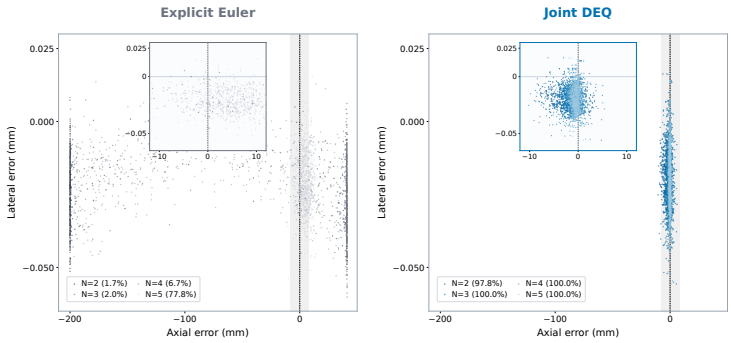


Figure 1: Panda reaching errors with explicit Euler (left) and joint implicit Euler (right), using 2–5 flow steps. Percentages show success over 640 paired trials per setting: final 3D error below 8 mm.

versus 1.7% for explicit Euler. This advantage persists at three to five steps, where implicit sampling achieves 100% success and explicit Euler reaches at most 77.8%. The comparison uses a fixed field without a trained neural policy.

Together, the analysis and measurements motivate implicit discretization. Its local sensitivity is governed by $(I - \Delta t\, J)^{-1}$ rather than $I + \Delta t\, J$, which can preserve concentration when the field contracts toward a target. The accuracy of implicit steps also depends on the conditioning of the underlying solve.

4 Method

We introduce a deep equilibrium flow policy (DEFlow), defined by implicit Euler integration and trained within soft actor–critic (SAC) (Haarnoja et al., 2018a;b). Following Sec. 3, we sample actions by solving the implicit Euler equations for all integration steps together. We compute policy gradients by implicit differentiation and action log-densities with the change-of-variables formula.

4.1 Implicit Flow Policy

For a state s, we transform a base sample $a_0 = z \sim p_0 = \mathcal{N}(0, \sigma_z^2 I)$ into an action through a state-conditioned velocity field $v_\theta(a, s, t)$. We take $\sigma_z > 0$, and the base distribution is independent of θ. We write its density as $p(z)$. We parameterize the velocity field using the gated form of Flow-G in SAC-Flow (Zhang et al., 2025),

$$v_\theta(a, s, t) = z_g(a, s, t) \odot \big(\tilde{h}(a, s, t) - a\big), \tag{6}$$

where $z_g \in (0, 1)^{d_a}$ is a sigmoid gate, $\tilde{h} \in \mathbb{R}^{d_a}$ is a scaled $\tanh$ output, and $\odot$ denotes elementwise multiplication. Both outputs depend on (a, s, t) and share parameters θ.

Let $\Delta t = 1/T$ and $t_k = k\Delta t$. We replace the explicit Euler rollout of SAC-Flow with implicit (backward) Euler (Hairer & Wanner, 1996):

$$\begin{aligned} a_k &= a_{k-1} + \Delta t\, v_\theta(a_k, s, t_k), \qquad k = 1, \dots, T, \\ a_{\text{env}} &= c \odot \tanh(a_T) + b. \end{aligned} \tag{7}$$

The field is evaluated at the new point a_k. The final squashing transformation follows SAC (Haarnoja et al., 2018b), with $c = (a_{\max} - a_{\min})/2 > 0$ and $b = (a_{\max} + a_{\min})/2$ enforcing the action bounds. Unless stated otherwise, we use $T = 4$ integration steps.

Implicit Euler is a standard choice for stiff dynamics (Hairer & Wanner, 1996). For the scalar linear test equation $\dot{x} = \lambda x$ with $\operatorname{Re}\lambda < 0$, its amplification factor $(1 - \Delta t\, \lambda)^{-1}$ has magnitude at most one for every $\Delta t > 0$, whereas explicit Euler requires $|1 + \Delta t\, \lambda| \le 1$. The finite-step analysis in Sec. 3 motivates our use of implicit Euler. For strongly contracting dynamics, explicit Euler may require very small integration steps to remain stable. In the scalar linear setting above, implicit Euler remains stable for any positive step size.

JOINT FIXED-POINT FORMULATION

Summing the first k implicit Euler updates in Eq. (7), with $a_0 = z$, expresses each intermediate action in terms of the base sample and the accumulated velocity:

$$a_k = z + \Delta t \sum_{j=1}^{k} v_\theta(a_j, s, t_j), \qquad k = 1, \dots, T. \tag{8}$$

We collect the intermediate actions into the vector $Z = (a_1^\top, \dots, a_T^\top)^\top \in \mathbb{R}^D$, where $D = Td_a$. Define $g_\theta(Z; s, z)$ by stacking the right-hand sides of Eq. (8) in the same order. The implicit Euler equations can then be written as a single fixed-point equation:

$$Z^\star = g_\theta(Z^\star; s, z). \tag{9}$$

The solution $Z^\star$ contains the intermediate actions $a_1^\star, \dots, a_T^\star$ satisfying all implicit Euler steps. Following the DEQ approach (Bai et al., 2019), we compute this solution with a numerical root finder.

The forward pass searches for a root of $g_\theta(Z; s, z) - Z = 0$. We use Anderson acceleration (Walker & Ni, 2011), initialized with every trajectory block equal to z, to approximate this root. Each evaluation of g_θ computes the velocities at the current estimates of the intermediate actions and accumulates them along the trajectory. The final block supplies a_T, which is squashed into the environment action by Eq. (7).

BACKWARD PASS

Following the implicit differentiation procedure of Bai et al. (2019, Eqs. (8) and (11)), we compute policy gradients by solving a linear system at the fixed point, without backpropagating through the root-finding iterations. For a differentiable sample loss $\ell(\theta, Z^\star)$, let $A = I - \partial_Z g_\theta$, with derivatives evaluated at $(Z^\star; s, z)$ for fixed (s, z). Assuming a differentiable solution branch and an invertible A, we solve for the adjoint vector $\lambda \in \mathbb{R}^D$ and evaluate the parameter gradient as

$$A^\top \lambda = \nabla_Z \ell, \qquad \nabla_\theta \ell = \partial_\theta \ell|_{Z^\star} + (\partial_\theta g_\theta)^\top \lambda. \tag{10}$$

Gradients are column vectors. The direct derivative $\partial_\theta \ell|_{Z^\star}$ holds the trajectory fixed and accounts for the explicit parameter dependence of the log-density and Jacobian penalty introduced below. The adjoint solve avoids forming A^{-1}.

In our implementation, we compute λ with a batched dense linear solve. Appendix A.2 gives the derivation, the structure of A, and full regularity assumptions. Appendix C.1 describes the finite-tolerance evaluation and numerical modifications.

4.2 MAXIMUM-ENTROPY POLICY OPTIMIZATION

To train the implicit flow policy with SAC, we need the log-density of its actions (Haarnoja et al., 2018b). We derive this density from the discrete implicit Euler steps and use it in the actor–critic updates below.

EXACT DENSITY BY DISCRETE CHANGE OF VARIABLES

To obtain the exact density we use the change-of-variables identity for invertible maps (Rezende & Mohamed, 2015; Papamakarios et al., 2021): for $x \mapsto y = f(x)$,

$$p_Y(y) = p_X\big(f^{-1}(y)\big)\,\big|\det \partial f^{-1}/\partial y\big| = p_X(x)/\big|\det \partial f/\partial x\big|, \tag{11}$$

Under the smoothness and invertibility assumptions in Appendix A.3, each implicit step in Eq. (7) defines a locally invertible map around the selected solution. Differentiating $a_k = a_{k-1} + \Delta t\, v_\theta(a_k, s, t_k)$ gives the differential $da_k = da_{k-1} + \Delta t\, J_k\, da_k$, so $(I - \Delta t\, J_k)\, da_k = da_{k-1}$ and the one-step Jacobian is

$$\frac{\partial a_k}{\partial a_{k-1}} = (I - \Delta t\, J_k)^{-1}, \qquad J_k = \left.\frac{\partial v_\theta}{\partial a}\right|_{(a_k, t_k)}. \tag{12}$$

Substituting Eq. (12) into Eq. (11), a single step changes the log-density by $\log p(a_k) = \log p(a_{k-1}) + \log\left|\det(I - \Delta t\, J_k)\right|$. The positive sign follows from the inverse in Eq. (12). Composing the T steps into the rollout $z = a_0 \to a_1 \to \cdots \to a_T$, the log-density is the base density plus one log-determinant per step. Applying the final squashing transformation gives the environment action $a_{\text{env}} = c \odot \tanh(a_T) + b$ (Eq. (7)). Its exact log-density is therefore:

$$\log \pi_\theta(a_{\text{env}} \mid s) = \log p(z) + \sum_{k=1}^{T} \log\left|\det(I - \Delta t\, J_k)\right| - \sum_{i=1}^{d_a} \log\left[c_i \left(1 - \tanh^2 a_{T,i}\right)\right] \tag{13}$$

Equation (13) gives the exact action log-density for the selected solution branch under the assumptions above. SAC-Flow uses a joint path log-density over intermediate actions as its entropy term (Zhang et al., 2025), whereas DIME uses a lower bound on the policy entropy (Celik et al., 2025). Here, the change-of-variables formula gives the log-density of the final action directly. The implementation uses finite solver tolerances, diagonal shifts, and clipping, which can modify the evaluated log-density and policy gradient. The full derivation is given in Appendix A.3.

Actor–Critic Updates

We train DEFlow within the SAC framework (Haarnoja et al., 2018b), using the action samples and log-densities derived above. Let $a_\theta(s, z) = c \odot \tanh(a_T^\star) + b$ denote an action sampled from the implicit policy. We compute the soft Bellman target as

$$y = r + \gamma \left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right], \tag{14}$$

where $a' = a_\theta(s', z')$, $z' \sim p_0$, and d is the termination indicator. Here $\bar{\phi}_j$ denotes the parameters of target critic j, and the action log-density is given by Eq. (13). For terminal transitions, we set $y = r$. Each critic is trained by minimizing the mean squared error

$$\mathcal{L}_Q(\phi_j) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D},\, z'\sim p_0}\left[\left(Q_{\phi_j}(s, a) - y\right)^2\right], \qquad j = 1, 2. \tag{15}$$

We update the implicit flow policy by minimizing the SAC actor objective (Haarnoja et al., 2018a;b):

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D},\, z\sim p_0}\left[\alpha \log \pi_\theta(a_\theta(s, z) \mid s) - \min_{j=1,2} Q_{\phi_j}(s, a_\theta(s, z))\right]. \tag{16}$$

The log-density is evaluated by Eq. (13). We compute the policy gradient using the implicit backward pass in Eq. (10). In the regularized configuration, we minimize $\mathcal{L}_\pi + \beta\mathcal{R}_J$, where $\beta \geq 0$ weights a per-step Jacobian penalty motivated by Jacobian regularization in DEQ models (Bai et al., 2021). Appendix B.1 defines $\mathcal{R}_J$ and specifies its hyperparameters.

The target critics are initialized with $\bar{\phi}_j = \phi_j$ and updated by exponential moving averages $\bar{\phi}_j \leftarrow (1 - \tau)\bar{\phi}_j + \tau\phi_j$ for $j = 1, 2$, where $\tau \in (0, 1]$ is the target averaging rate (Haarnoja et al., 2018b).

Algorithm

Algorithm 1 in Appendix B summarizes data collection and actor–critic updates following SAC (Haarnoja et al., 2018b). Each action $a_\theta(s, z)$ is obtained by solving Eq. (9) with Anderson acceleration and applying the squashing transformation in Eq. (7). Policy updates use the log-density in Eq. (13) and the implicit gradients in Eq. (10). The pseudocode takes the current temperature α and regularization weight β as inputs to each update. Numerical evaluation is described in Appendix C.1.

5 Experiments

5.1 Experimental Setup

We evaluate DEFlow in three settings: four variants of MultiGoal-4 with different navigation and docking tolerances (Haarnoja et al., 2017), five MuJoCo locomotion tasks (Todorov et al., 2012), and five manipulation tasks: PickCube and PushCube in ManiSkill (Tao et al., 2025), and Button-PressWall, PegInsertSide, and PushWall in MetaWorld (Yu et al., 2020). Training budgets are 200k

transitions for MultiGoal, 1M environment steps for MuJoCo, and 500k recorded steps for manipulation. Our main comparison methods are SAC (Haarnoja et al., 2018a), SAC-Flow, DIME, and QSM. For ablation studies, we compare DEFlow with a matched explicit-Euler flow on MultiGoal and vary the number of integration steps T. All baseline results come from our own implementations and training runs. Evaluation protocols, task-specific configurations, and additional statistical analyses are provided in Appendices B and C.

5.2 Benchmark Results

Successful diversity under precision constraints. We extend the four-goal MultiGoal task (Haarnoja et al., 2017) to test whether an expressive policy can combine diverse goal choices with precise actions. The agent receives a dense distance reward while navigating toward one of four symmetric goals. On entering a goal region, it performs one docking action and receives a bonus if the action is sufficiently close to the goal's target. We study four variants combining wide or narrow goal regions with loose or tight docking tolerances. Wide goal regions are circles of radius 1, while narrow regions are radial ellipses with semi-axes 1 and 0.20. Docking succeeds when the final action lies within distance 0.12 (loose) or 0.06 (tight) of the target associated with the reached goal. This design varies the precision required during navigation and during the final docking action separately. Figure 5 in Appendix C.2 illustrates the four variants. To maintain several successful strategies, the policy must choose different goals across episodes and concentrate its final action around the target associated with each goal.

Our main metric, successful coverage (U4), measures how many distinct goals a policy successfully completes within a small number of attempts. Let p_i be the probability that an episode from the origin reaches goal i and docks successfully. For four independent episodes under the same policy,

$$\mathrm{U4} = \frac{1}{4} \sum_{i=1}^{4} \left[1 - (1 - p_i)^4 \right]. \tag{17}$$

Here, $1 - (1 - p_i)^4$ is the probability of completing goal i at least once, so U4 is the expected fraction of distinct goals successfully completed. It rewards both reliable docking and coverage of alternative goals. Always succeeding at one goal gives $\mathrm{U4} = 0.25$, whereas uniform success across all four goals attains the maximum $1 - (3/4)^4 \approx 0.684$. We estimate p_i from 1,000 stochastic episodes per trained policy and average U4 across training seeds.

DEFlow achieves the highest mean successful coverage on all evaluated variants (Table 1). On loose docking, it approaches the balanced four-goal ceiling. Its mean successful coverage exceeds that of the second-best method by up to 17%. Thus, DEFlow retains the strongest combination of diversity and successful completion when the final action must be more precise. Figure 2 also shows that DEFlow develops successful coverage early in training. Appendix C.4 and Sec. 5.3 examine the failures of explicit flow and QSM.

Table 1: Successful coverage (U4) at 200k transitions: mean $\pm$ SD over completed seeds, 1,000 episodes per seed. Bold/underline mark the top two means.

Navigation	Docking	SAC	SAC-Flow	DIME	QSM	Explicit flow	DEFlow
Wide	Loose	0.397 ± 0.127	0.522 ± 0.224	$\underline{0.634} \pm 0.021$	0.000 ± 0.000	0.507 ± 0.030	$\mathbf{0.671} \pm 0.002$
Narrow	Loose	0.376 ± 0.019	0.613 ± 0.032	$\underline{0.632} \pm 0.012$	0.000 ± 0.000	0.499 ± 0.100	$\mathbf{0.661} \pm 0.020$
Wide	Tight	0.428 ± 0.105	0.481 ± 0.067	$\underline{0.502} \pm 0.011$	0.000 ± 0.000	0.412 ± 0.084	$\mathbf{0.589} \pm 0.128$
Narrow	Tight	0.467 ± 0.074	0.451 ± 0.084	$\underline{0.491} \pm 0.009$	0.000 ± 0.000	0.417 ± 0.153	$\mathbf{0.552} \pm 0.113$

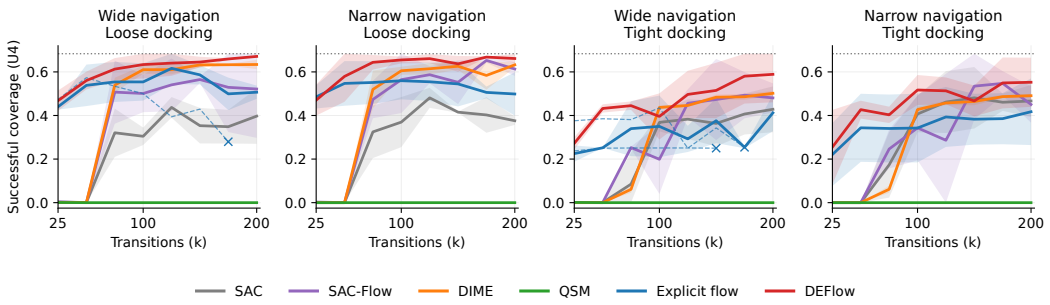


Figure 2: Successful coverage during training (mean ± SD). The dotted line marks the maximum U4.

Continuous control. Figure 3 shows learning curves across five MuJoCo tasks, and Table 6 in Appendix C.2 summarizes return at the recorded 1M evaluation. We report means and standard deviations across three runs per task. Appendix C.2 provides per-seed results, confidence intervals, and sensitivity analyses at alternative evaluation horizons. These tasks do not explicitly require multimodal action selection, and we observe no consistent advantage from greater policy expressivity, with SAC, SAC-Flow, DIME, and DEFlow achieving broadly comparable returns overall and SAC retaining a small lead in the sum of returns across tasks.

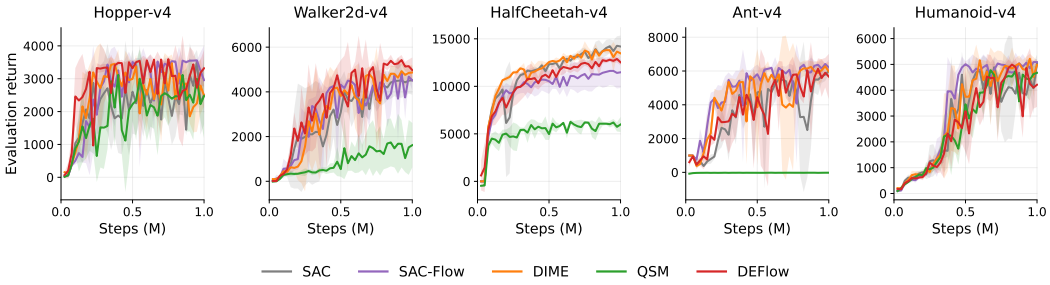


Figure 3: MuJoCo learning curves. Lines show run means, bands show ±1 sample SD.

Manipulation. Manipulation tasks test how quickly a policy turns contact with an object into useful behavior. We compare SAC, SAC-Flow, DIME, and DEFlow on five tasks from ManiSkill and MetaWorld, with a budget of 500k recorded training steps. Figure 4 shows deterministic evaluation returns, and Table 7 in Appendix C.2 summarizes their time average over training. Table 5 in Appendix C.2 reports final sampled ever-success: the fraction of evaluation episodes in which the policy completes the task at least once.

DEFlow's main advantage is early learning under the evaluated training recipes. It achieves the highest mean time-averaged return on four of the five tasks. On PickCube, the advantage also appears in task completion: DEFlow reaches higher sampled success at the intermediate checkpoints, before DIME catches up later in training. On PushCube, DEFlow attains high return by the first evaluation and maintains it through the rest of training. The return curves and early success evaluations thus capture gains that a final-checkpoint comparison alone would miss. At the final checkpoint, DIME has the highest sampled-success means on the cube tasks, while SAC leads on PegInsertSide.

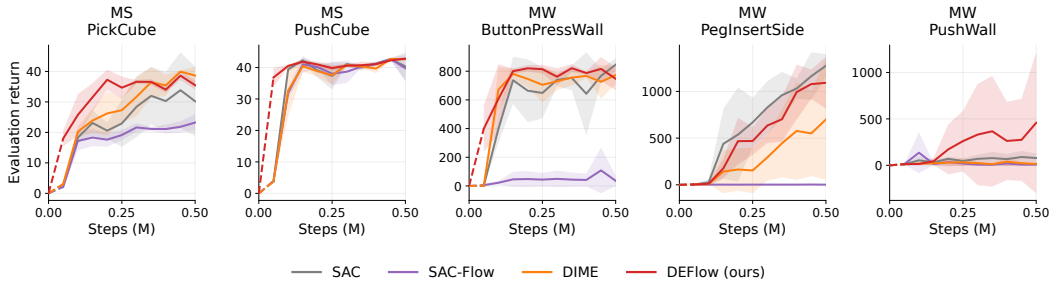


Figure 4: Manipulation evaluation returns, mean ± SD across three training seeds. Dashed segments connect an illustrative origin to the first evaluation.

5.3 Ablation Studies

Implicit vs. explicit flow. To test the role of implicit integration during policy learning, we replace DEFlow's implicit-Euler updates with explicit Euler while retaining the actor architecture, four integration steps, discrete density objective, and training settings. Both policies are trained independently on the docking tasks. DEFlow achieves higher successful coverage on loose docking (Table 1), and tight docking exposes a further limitation of the explicit variant: it either concentrates on a preferred goal or fails numerically (details in Appendix C.4). This comparison strongly supports implicit integration for learning diverse successful behavior under a narrow action tolerance.

Integrator step-count ablation. On the MultiGoal variant with wide goal regions and tight docking, we freeze two policies trained with $T = 4$ and change only the number of docking integration steps. Each policy keeps its own integrator and uses the same states and Gaussian inputs across step counts. At the training depth $T = 4$, the explicit policy achieves only 50.1% balanced docking accuracy on a shared bank of reachable states, with near-zero accuracy for two of the four goals, while DEFlow achieves 94.1%. Even on each policy's own visited states, changing only the docking integration depth from $T = 4$ to $T = 2$ reduces explicit success from 99.8% to 0.7%, versus 96.5% to 53.7% for DEFlow, and increasing T also degrades the explicit policy. For this pair, DEFlow is more robust to changes in step count (Figures 8 and 9).

6 Limitations and Costs

Training time. In our measured runs, DEFlow had roughly 40% lower effective training throughput than SAC-Flow and took about three times as long as SAC. Its wall-clock cost was of the same order as DIME's. These comparisons are approximate and depend on workload and shared hardware. Each implicit actor update requires a nonlinear forward solve and a dense linear backward solve.

Exactness is tolerance-limited in training. Equation (13) is exact for a solution of the implicit equations under the assumptions in Appendix A.3. The implementation uses finite solver tolerances and numerical modifications (Appendix C.1). The trained-policy audit in Appendix C.1 measures the resulting discrepancies at final manipulation checkpoints. The audit finds numerical inaccuracy, especially in PushWall actor gradients. Tightening the solver at evaluation reduces these errors, while sampled task success changes little. We use the original solver settings for benchmark evaluations to avoid the extra cost of tighter solves.

7 Conclusion

We introduced DEFlow, a deep-equilibrium formulation of flow policies for maximum-entropy reinforcement learning. By combining implicit sampling, tractable density evaluation, and implicit differentiation, DEFlow provides an alternative to explicit flow actors. Our experiments highlight its potential for expressive control and the remaining trade-offs in numerical accuracy and computational cost.

REFERENCES

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019. doi: 10.48550/arXiv.1909.01377. URL https://arxiv.org/abs/1909.01377.

Shaojie Bai, Vladlen Koltun, and J. Zico Kolter. Stabilizing equilibrium models by jacobian regularization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 554–565. PMLR, 2021. URL https://proceedings.mlr.press/v139/bai21b.html.

Onur Celik, Zechu Li, Denis Blessing, Ge Li, Daniel Palenicek, Jan Peters, Georgia Chalvatzaki, and Gerhard Neumann. DIME: Diffusion-based maximum entropy reinforcement learning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 6958–6977. PMLR, 2025. URL https://proceedings.mlr.press/v267/celik25a.html.

Zihan Ding and Chi Jin. Consistency models as a rich and efficient policy class for reinforcement learning. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2309.16984. URL https://arxiv.org/abs/2309.16984.

Alan Edelman, Ekin Akyürek, and Yuyang Wang. Backpropagation through back substitution with a backslash. *SIAM Journal on Matrix Analysis and Applications*, 45(1):429–449, 2024. doi: 10.1137/22M1532871. URL https://doi.org/10.1137/22M1532871.

Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1352–1361. PMLR, 2017. URL https://proceedings.mlr.press/v70/haarnoja17a.html.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870. PMLR, 2018a. URL https://proceedings.mlr.press/v80/haarnoja18b.html.

Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications, 2018b. URL https://arxiv.org/abs/1812.05905.

Ernst Hairer and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, volume 14 of *Springer Series in Computational Mathematics*. Springer, Berlin, Heidelberg, 2 edition, 1996. doi: 10.1007/978-3-642-05221-7.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.02747.

George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

Seohong Park, Qiyang Li, and Sergey Levine. Flow q-learning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48104–48127. PMLR, 2025. URL https://proceedings.mlr.press/v267/park25f.html.

Michael Psenka, Alejandro Escontrela, Pieter Abbeel, and Yi Ma. Learning a diffusion model policy from rewards via q-score matching. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 41163–41182. PMLR, 2024. URL https://proceedings.mlr.press/v235/psenka24a.html.

Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pp. 1530–1538, 2015.

Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-Kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Demonstrating gpu parallelized robot simulation and rendering for generalizable embodied ai with maniskill3. In *Proceedings of Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.021. URL https://www.roboticsproceedings.org/rss21/p021.html.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109. URL https://doi.org/10.1109/IROS.2012.6386109.

Homer F. Walker and Peng Ni. Anderson acceleration for fixed-point iterations. *SIAM Journal on Numerical Analysis*, 49(4):1715–1735, 2011. doi: 10.1137/10078356X. URL https://doi.org/10.1137/10078356X.

Zhendong Wang, Jonathan J. Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2208.06193. URL https://arxiv.org/abs/2208.06193.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 1094–1100. PMLR, 2020. URL https://proceedings.mlr.press/v100/yu20a.html.

Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Mo Guang, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling, 2025. URL https://arxiv.org/abs/2509.25756.

A Additional Derivations

A.1 Finite-Step Analysis

This appendix records the assumptions and proof details behind the claims in Sec. 3. We use $T \in \mathbb{N}_+$ integration steps on $[0, 1]$ with $\Delta t = 1/T$ and $t_k = k\Delta t$. We fix a state and suppress it from the notation. The actions are unsquashed, the base variable is shared when two rollouts are compared, and the field is the ideal independent-interpolation field. These results isolate the numerical effects of finite-step integration on analytically specified vector fields.

A smooth multimodal target can require a fine grid

Consider

$$A = mC + \sigma G, \qquad C \in \{-1, 1\}, \qquad I_t = (1-t)Z + tA, \tag{18}$$

where $m, \sigma > 0$, $\mathbb{P}(C = \pm 1) = 1/2$, and Z, G are standard Gaussians. The variables C, Z, G are mutually independent. Small-width limits keep m fixed. The terminal target is $\pi^\star_{m,\sigma} = \frac{1}{2}\mathcal{N}(-m, \sigma^2) + \frac{1}{2}\mathcal{N}(m, \sigma^2)$. Writing $q_t = (1-t)^2 + \sigma^2 t^2$, conditional Gaussian regression gives

$$v^\star(x, t) = b_t x + d_t \tanh(k_t x), \quad b_t = \frac{\sigma^2 t - (1-t)}{q_t}, \quad d_t = \frac{m(1-t)}{q_t}, \quad k_t = \frac{mt}{q_t}. \tag{19}$$

The denominator is bounded away from zero, so this field is globally well posed. Its derivative is

$$\partial_x v^\star(x, t) = b_t + d_t k_t \operatorname{sech}^2(k_t x). \tag{20}$$

For $0 < \sigma \le \min\{1/2, m/10\}$, evaluating at $x = 0$ and $t = (1+\sigma)^{-1}$ yields

$$\sup_{x,t} |\partial_x v^\star(x, t)| \ge \frac{m^2}{8\sigma^3}. \tag{21}$$

Thus the exact field remains regular for every fixed $\sigma > 0$, while the near-identity requirement for a discrete step becomes increasingly demanding as the target narrows.

For completeness, let $J_k = D_a v(a_k, t_k)$ and $B_k = I + \Delta t\, J_k$. If $\|J_k\| \le L_k$, the triangle and reverse-triangle inequalities give

$$[1 - \Delta t\, L_k]_+ \le \sigma_{\min}(B_k) \le \sigma_{\max}(B_k) \le 1 + \Delta t\, L_k. \tag{22}$$

When $\Delta t\, L_k < 1$ for all k, multiplication of singular-value bounds gives

$$\kappa_2(B_{T-1} \cdots B_0) \le \prod_{k=0}^{T-1} \frac{1 + \Delta t\, L_k}{1 - \Delta t\, L_k}. \tag{23}$$

The same calculation for an implicit step produces the inverse factor $(I - \Delta t\, J_{k+1})^{-1}$, which is why the two discretizations can have different conditioning on the same field.

Sampling error. For the explicit Euler discretization of (19), define $\pi^\star_{\Delta t}$ as the law after $T = 1/\Delta t$ steps. The linear part of the update contracts the outward fluctuation. An induction gives

$$|\widehat{X}_k| \le m t_k + R_k |Z|, \qquad R_{k+1} = (1 + \Delta t\, b_{t_k})\, R_k, \quad R_0 = 1. \tag{24}$$

Convexity of $\sqrt{q_t}$ implies

$$R_T \le \frac{\sigma^2(1 - \Delta t)}{\sqrt{(\Delta t)^2 + \sigma^2(1 - \Delta t)^2}} \le \frac{\sigma^2}{\Delta t}. \tag{25}$$

Now use the 1-Lipschitz function $f_m(x) = (|x| - m)_+$. The envelope gives $\|f_m(\widehat{X}_T)\|_{L^2} \le R_T$, while the target satisfies $\|f_m(A)\|_{L^2} \ge \sigma/\sqrt{2}$. For any coupling of the numerical and target variables, the reverse triangle inequality in L^2 therefore gives

$$W_2(\pi^\star_{\Delta t}, \pi^\star_{m,\sigma}) \ge \left[\frac{\sigma}{\sqrt{2}} - R_T\right]_+. \tag{26}$$

Substitution of (25) proves

$$\Delta t \ge 2\sqrt{2}\,\sigma \quad \Longrightarrow \quad W_2(\pi^\star_{\Delta t}, \pi^\star_{m,\sigma}) \ge \frac{\sigma}{2\sqrt{2}}. \tag{27}$$

At one step, $v^\star(x, 0) = -x$, so $\widehat{X}_1 = 0$ and $W_2(\pi^\star_1, \pi^\star_{m,\sigma}) = \sqrt{m^2 + \sigma^2}$. At two steps, direct substitution gives

$$\widehat{X}_2 = \frac{\sigma^2 Z + m \tanh\big(mZ/(1 + \sigma^2)\big)}{1 + \sigma^2}. \tag{28}$$

As $\sigma \downarrow 0$, this converges in L^2 to $m \tanh(mZ)$, whereas the target converges to the two-point law on $\{-m, m\}$. The limiting Wasserstein error is consequently

$$m \left(\mathbb{E}\left[(1 - \tanh(m|Z|))^2\right]\right)^{1/2} > 0. \tag{29}$$

This establishes intermodal shape distortion without invoking a singular explicit update.

A FIXED BUDGET CAN DISTORT THE POLICY GRADIENT

The gradient statement in Sec. 3 uses the centered Gaussian subfamily $A \sim \mathcal{N}(0, \vartheta)$, with $\sigma = \sqrt{\vartheta}$, and the terminal loss $\ell(a) = a^2/2$. The exact flow gives $X_1 = \sqrt{\vartheta} Z$ and $\mathcal{L}'(\vartheta) = 1/2$. For $T \ge 2$, direct substitution of $t_k = k/T$ into the explicit update yields

$$R_T(\vartheta) = \prod_{k=0}^{T-1} \frac{(T - k)(T - k - 1) + \vartheta k(k + 1)}{(T - k)^2 + \vartheta k^2}, \qquad \widehat{X}_T = R_T(\vartheta) Z. \tag{30}$$

Hence $\mathcal{L}'_{\Delta t}(\vartheta) = R_T R'_T$. Setting $u = T^2 \vartheta = (\sigma/\Delta t)^2$, the factors are nondecreasing and their logarithmic derivatives give

$$0 \le 2 R_T R'_T \le 2u e^{2u}(1 + u) \le 3u, \qquad 0 \le u \le \frac{1}{8}. \tag{31}$$

Since the exact derivative is $1/2$, this is the claimed bound on the relative gradient. The statement is deliberately limited to this terminal loss and parameterization; it identifies a possible rollout-sensitivity mechanism rather than characterizing the full SAC actor gradient.

WHY ARCHITECTURAL STABILIZATION ALONE IS INSUFFICIENT

Two short counterexamples clarify the scope of the preceding results. First, clipping an optimizer step bounds $\|\theta^{r+1} - \theta^r\|$, but does not bound $\|D_a v_\theta\|$, the smallest singular value of a step map, or the sampling error. For $v_\lambda(x) = -\lambda x$, explicit Euler gives

$$X_1 = e^{-\lambda} Z, \qquad \widehat{X}_T = (1 - \Delta t\lambda)^T Z, \qquad W_2 = \left| |1 - \Delta t\lambda|^T - e^{-\lambda} \right|. \tag{32}$$

The continuous flow is well posed, while $\lambda = 1/\Delta t$ makes one step singular and $\lambda > 2/\Delta t$ amplifies its sensitivity. A clipped update can be zero at a parameter-space minimizer, so clipping does not exclude this field.

Second, a bounded gated field can keep every Euler state in a bounded range while its gate derivative makes the step Jacobian singular or arbitrarily ill-conditioned. Normalized features and a bounded output head do not remove this possibility. These examples show why the method must expose and control the discrete step structure; they do not imply that the trained actor always enters the counterexample regime.

A.2 IMPLICIT DIFFERENTIATION

We derive the backward pass in Sec. 4.1 following Bai et al. (2019). Fix (s, z) and let $Z^\star$ solve Eq. (9). Assume that the velocity field has continuous first and second derivatives in (a, θ) and that each matrix $I - \Delta t J_k$ is invertible at this solution, where $J_k = \partial_a v_\theta(a_k^\star, s, t_k)$.

Differentiating the fixed-point equation gives

$$A \frac{dZ^\star}{d\theta} = \partial_\theta g_\theta, \qquad A = I - \partial_Z g_\theta. \tag{33}$$

All derivatives are evaluated at $(Z^\star; s, z)$. From Eq. (8), block k depends only on actions $1, \dots, k$. The blocks of A are therefore

$$A_{kj} = \begin{cases} -\Delta t J_j, & j < k, \\ I - \Delta t J_k, & j = k, \\ 0, & j > k. \end{cases} \qquad \det A = \prod_{k=1}^{T} \det(I - \Delta t J_k). \tag{34}$$

Thus A is block lower triangular and invertible under the stated assumptions. The implicit function theorem then gives a locally differentiable solution as (z, θ) varies.

For a differentiable sample loss $\ell(\theta, Z^\star)$, the chain rule gives

$$\nabla_\theta \ell = \partial_\theta \ell|_{Z^\star} + \left(\frac{dZ^\star}{d\theta} \right)^\top \nabla_Z \ell = \partial_\theta \ell|_{Z^\star} + (\partial_\theta g_\theta)^\top A^{-\top} \nabla_Z \ell. \tag{35}$$

Solving $A^\top \lambda = \nabla_Z \ell$ recovers Eq. (10). This avoids forming either A^{-1} or the full derivative $dZ^\star/d\theta$. The direct term $\partial_\theta \ell|_{Z^\star}$ includes the dependence of the log-density and Jacobian penalty on θ with the actions held fixed. Their derivatives involve second derivatives of the velocity field.

To apply this calculation to the expected actor loss, we assume that, for almost every (s, z), the selected solution remains differentiable throughout a common neighborhood of the current parameters. We also assume that the sample-loss derivatives in this neighborhood are bounded by a function of (s, z) with finite expectation. Holding the replay distribution fixed, these conditions allow differentiation under the expectation and justify the minibatch gradient in Algorithm 1. Numerical evaluation of this gradient is described in Appendix C.1.

A.3 ACTION LOG-DENSITY

We justify the change-of-variables calculation in Sec. 4.2. Fix a state s. Assume that a measurable, deterministic rule selects an implicit trajectory for almost every $z \sim p_0$. On open regions covering these samples up to a set of probability zero, assume that the selected trajectory is continuously differentiable in z and that every $I - \Delta t J_k$ is invertible.

The log form of the change-of-variables identity in Eq. (11) is

$$\log p_Y(y) = \log p_X(x) - \log \left| \det \partial f / \partial x \right|. \tag{36}$$

An implicit Euler step has the explicit inverse relation

$$a_{k-1} = H_k(a_k), \qquad H_k(a) = a - \Delta t\, v_\theta(a, s, t_k). \tag{37}$$

Consequently, the endpoint determines every preceding action and the base sample: $z = H_1 \circ \cdots \circ H_T(a_T)$. Two different base samples cannot produce the same selected endpoint. The selected map $z \mapsto a_T^\star$ is therefore one-to-one, even if the step equations admit other solutions. The pre-squash density is defined on the image of this selected map and is zero outside it.

Since $\partial_a H_k = I - \Delta t J_k$, differentiating the inverse map gives

$$\frac{\partial z}{\partial a_T} = (I - \Delta t J_1) \cdots (I - \Delta t J_T). \tag{38}$$

Composing the step maps and applying the log form of change of variables gives

$$\frac{\partial a_T}{\partial z} = (I - \Delta t\, J_T)^{-1} \cdots (I - \Delta t\, J_1)^{-1}, \qquad \log \pi(a_T \mid s) = \log p(z) + \sum_{k=1}^{T} \log\bigl|\det(I - \Delta t\, J_k)\bigr|. \tag{39}$$

Here, $\pi(a_T \mid s)$ denotes the density before squashing. The final transformation $a_{\text{env}} = c \odot \tanh(a_T^\star) + b$ is one-to-one for $c_i > 0$, with Jacobian determinant $\prod_i c_i(1 - \tanh^2(a_{T,i}^\star))$. Dividing by this determinant and taking logarithms gives the final-action log-density:

$$\log \pi_\theta(a_{\text{env}} \mid s) = \log p(z) + \sum_{k=1}^{T} \log\bigl|\det(I - \Delta t\, J_k)\bigr| - \sum_{i=1}^{d_a} \log\bigl[c_i\,(1 - \tanh^2 a_{T,i})\bigr]$$

This is Eq. (13), including the action-bound correction used in SAC (Haarnoja et al., 2018b). This identity concerns an exact solution of the implicit equations. Appendix C.1 specifies how it is evaluated at finite solver tolerance.

B Implementation Details

B.1 Architecture

DEFlow uses the gated velocity field of SAC-Flow (Zhang et al., 2025). Its inputs are the observation, intermediate action, and time embedding. The time embedding maps 32 sinusoidal features through two layers of widths 64 and 32. A shared projection with SiLU and layer normalization feeds separate gate and candidate heads. Each head has one hidden layer and outputs an action-dimensional vector. The gate is sigmoid, and the candidate is a scaled $\tanh$ as in Eq. (6). The velocity width is 256, the candidate scale is 4, and the gate output bias is initialized to 1. Actions start from standard Gaussian noise and use four implicit-Euler steps before the final $\tanh$ and action-bound scaling.

Jacobian regularization. Motivated by Jacobian regularization for DEQ models (Bai et al., 2021), we penalize the individual velocity Jacobians:

$$\mathcal{R}_J(\theta) = \mathbb{E}_{s\sim\mathcal{D},\, z\sim p_0}\left[\sum_{k=1}^{T}\left[\|\Delta t J_k\|_2 - \sigma^*\right]_+^2\right], \qquad [x]_+ = \max(x, 0), \tag{40}$$

and minimize $\mathcal{L}_\pi + \beta\mathcal{R}_J$. We use $\sigma^* = 0.9$ and increase β from zero to 0.01 over the first $100{,}000$ environment steps.

Temperature updates. In the regularized DEFlow recipe, we update $\log\alpha$ from the actor log-densities. The target is an exponential moving average of negative log-density (weight 0.01) minus 0.7, with floor $-0.7d_a$. The update winsorizes log-densities at the minibatch 5th and 95th percentiles, leaks $\log\alpha$ toward $\log(0.05)$ with weight 0.01, and enforces $\alpha \geq 0.02$. We also tested fixed-temperature variants, which sometimes reached comparable return but did not provide stable adaptive temperature behavior.

B.2 Implementation Details

Algorithm 1 summarizes the SAC updates with DEFlow's implicit actor. Table 2 gives one reference configuration for the matched-critic MuJoCo comparison. All methods use twin 4×512 ReLU critics in this comparison. We retain each baseline's policy class and actor objective: SAC has a squashed Gaussian density (Haarnoja et al., 2018b), SAC-Flow uses its noisy explicit rollout and joint-path entropy term (Zhang et al., 2025), DIME uses its diffusion entropy bound (Celik et al., 2025), and QSM matches its score to the critic action gradient without a policy log-density (Psenka et al., 2024). We implement each baseline's actor and objective in a common training pipeline. Matching critic capacity is necessary to assess actor performance without confounding it with differences in the critic.

Table 2: Reference matched-critic MuJoCo configuration. Method rows retain actor-specific sampling and objectives; full run settings are in the repository.

Method	Actor steps	Policy objective or density
SAC	—	Gaussian log-density
SAC-Flow	4	Joint-path entropy
DIME	16	Diffusion entropy bound
QSM	5	Q-score matching (no log-density)
DEFlow	4	Discrete change-of-variables density
Shared: twin 4×512 critics; batch 512; replay 10^6; warm-up 50k; UTD 1. $\gamma = 0.99$; $\tau = 0.005$; actor/critic learning rates $3 \times 10^{-4}/10^{-3}$.		

B.3 Benchmark Protocol

The benchmarks comprise MultiGoal-4 (Haarnoja et al., 2017), five MuJoCo locomotion tasks (Hopper-v4, Walker2d-v4, HalfCheetah-v4, Ant-v4, Humanoid-v4) (Todorov et al., 2012), two Man-

Algorithm 1 DEFlow

Require: Initial policy parameters θ, critic parameters ϕ_1, ϕ_2, minibatch size B, target averaging rate τ.

Initialize target parameters $\bar{\phi}_j \leftarrow \phi_j$ for $j = 1, 2$ and replay buffer $\mathcal{D} \leftarrow \varnothing$.
Reset the environment and observe the initial state s.
for each iteration **do**
  for each environment step **do**
    Draw $z \sim p_0$ and compute $a = a_\theta(s, z)$.
    Execute a and observe reward r, next state s', and termination indicator d.
    Store (s, a, r, s', d) in $\mathcal{D}$.
    Set $s \leftarrow s'$, or reset the environment if the episode has ended.
  end for
  for each gradient step **do**
    Draw a minibatch $\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{B}$ from $\mathcal{D}$.
    Draw $z'_i \sim p_0$ and compute $a'_i = a_\theta(s'_i, z'_i)$ for each i.
    Evaluate $\log \pi_\theta(a'_i \mid s'_i)$ using Eq. (13).
    Compute targets y_i using Eq. (14), with $y_i = r_i$ when $d_i = 1$.
    Update ϕ_1, ϕ_2 by minimizing the minibatch critic loss in Eq. (15), holding y_i fixed.
    if a policy update is due **then**
      Draw fresh $z_i \sim p_0$ and compute $\tilde{a}_i = a_\theta(s_i, z_i)$ for each i.
      Evaluate $\log \pi_\theta(\tilde{a}_i \mid s_i)$ using Eq. (13).
      Form each sample loss ℓ_i using Eq. (16) and the β-weighted penalty in Eq. (40).
      Solve the adjoint systems in Eq. (10) to compute $\nabla_\theta \ell_i$.
      Update θ using $B^{-1} \sum_{i=1}^{B} \nabla_\theta \ell_i$.
      Update α.
    end if
    Update target parameters $\bar{\phi}_j \leftarrow (1 - \tau)\bar{\phi}_j + \tau\phi_j$ for $j = 1, 2$.
  end for
end for

iSkill tasks (PickCube-v1 and PushCube-v1) (Tao et al., 2025), and three MetaWorld tasks (Button-PressWall, PegInsertSide, PushWall) (Yu et al., 2020). All methods use the same task wrapper and checkpoint schedule within each comparison. MuJoCo and manipulation runs use budgets of 1M and 500k recorded steps, respectively. Manipulation wrappers set task-specific rewards, resets, horizons, and termination rules; their exact settings are provided in the repository's run configurations.

MuJoCo returns use the recorded evaluation at 1M steps, curves use evaluations every 25k steps. Manipulation returns are evaluated from 50k to 500k steps and averaged over the recorded checkpoints for the time-averaged-return table. We report means and sample standard deviations across three training seeds per task and method. For deterministic DEFlow evaluation, the base sample is $z = 0$, QSM retains stochastic denoising even in its evaluation channel. Final manipulation ever-success is the fraction of sampled episodes in which the task succeeds at least once, measured over 100 episodes per trained policy. These evaluation episodes are not additional training seeds. The exact selected runs and evaluation scripts are linked from the repository's result manifests.

B.4 Fixed-Flow Robot Illustration

Figure 1 isolates discretization in a fixed scalar contraction, $v(x, t) = -8(x - 0.8)$, on $t \in [0, 1]$, initialized with $z \sim \mathcal{N}(0, 1)$. We compare explicit Euler with a jointly solved implicit-Euler trajectory at $T \in \{2, 3, 4, 5\}$ and apply $u = \tanh(x_T)$ to the terminal sample. The implicit solver uses damped Newton with zero initialization, an 80-iteration cap, and normalized local and cumulative fixed-point residual tolerances of 10^{-11} and 10^{-9}, respectively, in double precision. This diagnostic has no learned velocity field, critic, or policy training; the learned DEFlow policies in Sec. 4 use Anderson acceleration.

The simulator is a custom one-decision Panda waypoint-reaching task in ManiSkill 3. The scalar command specifies a displacement of $0.12u$ metres along the task axis from the calibrated end-effector anchor. The target uses $u_\star = \tanh(0.8(1 - e^{-8}))$, the continuous-flow image of $z = 0$ after

squashing. Each waypoint is held for 40 Cartesian position-control steps at 20 Hz, with the gripper fixed and no early success termination. Success is the final three-dimensional end-effector distance to the target being less than 8 mm; it is not an ever-success metric. The plot shows axial and lateral projections, while its success labels use all three coordinates.

Each method and step count uses 640 trials: 128 trials for each of seeds $0, 1, 2, 3, 4$. Simulator states and Gaussian base samples are shared across methods and step counts. The figure reuses recorded endpoints from the descriptive step-count sweep and is not an independent replication. The comparison holds the number of integration steps fixed, without matching field evaluations or wall-clock time. It supports a discretization comparison for this contraction task, without establishing computational efficiency, multimodal sampling accuracy, or performance of a trained policy.

C Additional Results

C.1 Numerical Verification

DEFlow solves the joint trajectory with Anderson acceleration (Walker & Ni, 2011). The solver stops when the minibatch mean relative residual meets its configured tolerance or the iteration limit is reached, returning the last iterate in either case. Actor updates re-evaluate the field at that iterate and use a dense adjoint solve rather than differentiating through solver iterations. The adjoint and log-determinant matrices receive a $10^{-6}I$ shift, and log-determinants are clipped to $[-50, 50]$. Some training recipes also clip actor log-densities to $[-15, 15]$. The reference MuJoCo configuration uses a 3×10^{-4} forward tolerance.

We test how the forward solver affects trained DEFlow policies by comparing each final manipulation policy under two settings: the original Anderson solver with a 25-iteration limit and 10^{-5} tolerance, and a tighter solver with a 100-iteration limit and 10^{-6} tolerance. Table 3 compares actions, safeguarded log-density readouts, and actor gradients with an independently solved implicit reference; the median and P95 pool matched inputs within each task. Three PushWall inputs without a validated reference are excluded from these discrepancies. Table 4 reports stochastic ever-success on paired environment episodes as the mean $\pm$ sample standard deviation over three training seeds. Both settings use the same 100 evaluation episodes per checkpoint, and the change is Tighter minus Original in percentage points.

Table 3: Numerical discrepancies on trained policies.

		Original		Tighter	
Task	Discrepancy	Median	P95	Median	P95
PickCube	Action	1.49×10^{-5}	1.62×10^{-4}	7.04×10^{-7}	2.39×10^{-6}
	Log-density (nat)	3.73×10^{-5}	5.18×10^{-4}	2.05×10^{-6}	9.92×10^{-6}
	Gradient (%)	0.00330	0.771	0.000143	0.000980
PushCube	Action	1.02×10^{-5}	3.20×10^{-5}	9.73×10^{-7}	2.38×10^{-6}
	Log-density (nat)	2.03×10^{-5}	8.15×10^{-5}	2.47×10^{-6}	8.91×10^{-6}
	Gradient (%)	0.000930	0.00465	0.000229	0.000642
ButtonPressWall	Action	1.63×10^{-5}	9.92×10^{-4}	9.95×10^{-7}	3.10×10^{-6}
	Log-density (nat)	4.49×10^{-5}	1.91×10^{-3}	2.55×10^{-6}	1.05×10^{-5}
	Gradient (%)	0.00630	0.836	0.000420	0.00178
PegInsertSide	Action	8.54×10^{-6}	2.75×10^{-5}	9.69×10^{-7}	2.57×10^{-6}
	Log-density (nat)	2.22×10^{-5}	1.31×10^{-4}	2.09×10^{-6}	9.18×10^{-6}
	Gradient (%)	0.000523	0.00435	0.000153	0.000390
PushWall	Action	0.00299	0.0375	0.000479	0.0211
	Log-density (nat)	0.0108	0.299	0.00122	0.197
	Gradient (%)	13.7	96.1	2.84	50.3

Table 4: Final sampled success under each solver.

Task	Original	Tighter	Change (pp)
PickCube	87.0 ± 8.9	86.7 ± 8.7	-0.33
PushCube	98.7 ± 1.2	99.0 ± 1.0	$+0.33$
ButtonPressWall	8.7 ± 14.2	8.3 ± 13.6	-0.33
PegInsertSide	38.0 ± 23.8	38.0 ± 23.8	0.00
PushWall	0.3 ± 0.6	0.3 ± 0.6	0.00

The tighter settings reduce the median and P95 discrepancies for every task and quantity in Table 3. Final sampled success changes little on the paired episodes in Table 4. This means that a more accurate solver at evaluation does not improve task success overall. For the iteration limit, the fraction of evaluation calls that reach it averages 47.2% with Original and 13.1% with Tighter across checkpoints. These results measure the sensitivity of final policies to the tested solver change, while errors during learning and the likelihood of the finite-iteration sampler remain open. Original is reevaluated in the same runtime as Tighter and can differ from the earlier benchmark evaluation.

C.2 Extended Benchmark Results

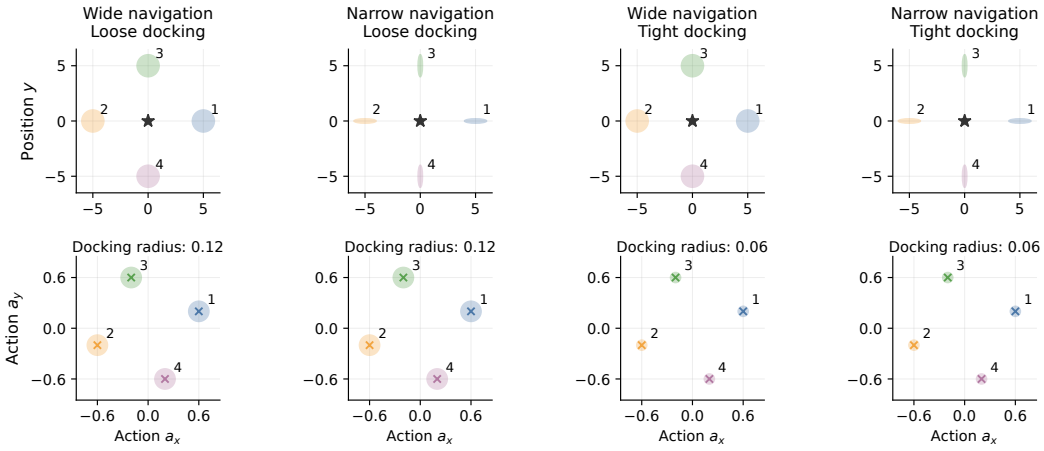


Figure 5: Four MultiGoal docking variants. Top: navigation goal regions. Bottom: corresponding docking targets and action tolerances; colors and numbers identify goals.

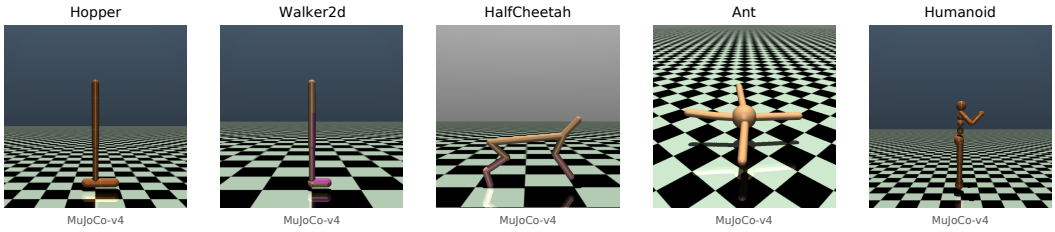


Figure 6: MuJoCo continuous-control environments used in our experiments.

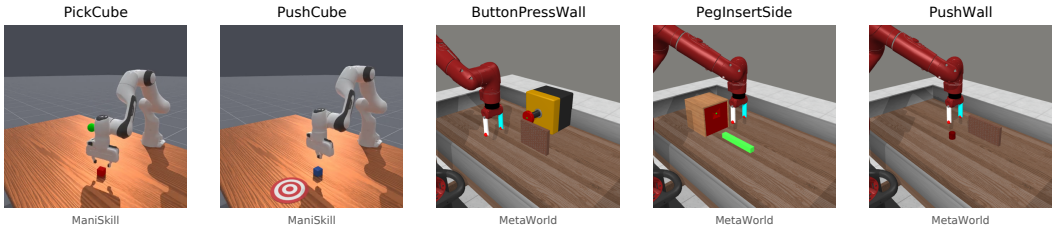


Figure 7: Manipulation tasks from ManiSkill (first two) and MetaWorld (last three). ManiSkill images are from the official task demonstrations.

Table 5: Final sampled ever-success (%) at 500k steps, mean ± SD across three training seeds. Bold and underlined values mark the top two means.

Task	SAC	SAC-Flow	DIME	DEFlow
PickCube	64.7 ± 56.0	5.0 ± 8.7	$\mathbf{93.0 \pm 9.5}$	$\underline{87.3 \pm 8.3}$
PushCube	97.0 ± 3.6	97.3 ± 3.1	$\mathbf{99.0 \pm 1.7}$	$\underline{98.7 \pm 1.2}$
ButtonPressWall	0.0 ± 0.0	0.0 ± 0.0	$\underline{3.7 \pm 2.1}$	$\mathbf{8.7 \pm 14.2}$
PegInsertSide	$\mathbf{72.3 \pm 9.8}$	0.0 ± 0.0	23.7 ± 41.0	$\underline{39.3 \pm 21.5}$
PushWall	0.0 ± 0.0	$\mathbf{10.3 \pm 17.9}$	$\underline{1.0 \pm 1.7}$	0.3 ± 0.6
Sum	$\underline{234.0}$	112.7	220.3	**234.3**

Table 6: MuJoCo return at 1M steps, mean ± SD across three runs. Bold and underlined values mark the highest and second-highest means.

Environment	SAC	SAC-Flow	DIME	QSM	DEFlow
Hopper	2510 ± 901	$\underline{2955 \pm 1055}$	2485 ± 326	2474 ± 1284	$\mathbf{3318 \pm 560}$
Walker2d	4517 ± 644	4513 ± 436	$\underline{4866 \pm 348}$	1615 ± 992	$\mathbf{4973 \pm 564}$
HalfCheetah	$\mathbf{14190 \pm 1152}$	11476 ± 1647	$\underline{13476 \pm 527}$	5981 ± 791	12534 ± 340
Ant	$\underline{5995 \pm 188}$	$\mathbf{6215 \pm 458}$	5975 ± 568	-22 ± 2	5673 ± 1340
Humanoid	$\underline{5042 \pm 334}$	$\mathbf{5089 \pm 123}$	4981 ± 185	4672 ± 191	4211 ± 858
Sum	**32253**	30247	$\underline{31783}$	14721	30709

Table 7: Time-averaged return over 50k–500k steps, mean ± SD across three seeds. Bold and underlined values mark the top two means.

Environment	SAC	SAC-Flow	DIME	DEFlow
MS PickCube	25.1 ± 5.9	19.0 ± 1.5	$\underline{29.1 \pm 3.0}$	$\mathbf{33.5 \pm 0.5}$
MS PushCube	$\underline{38.4 \pm 0.7}$	37.3 ± 1.2	37.3 ± 1.5	$\mathbf{40.8 \pm 0.2}$
MW ButtonPressWall	641.6 ± 91.1	47.2 ± 55.6	$\underline{696.7 \pm 30.1}$	$\mathbf{755.7 \pm 24.9}$
MW PegInsertSide	$\mathbf{699.1 \pm 209.1}$	2.5 ± 0.0	298.2 ± 237.4	$\underline{564.3 \pm 158.7}$
MW PushWall	$\underline{61.2 \pm 39.3}$	28.1 ± 38.5	23.4 ± 19.4	$\mathbf{217.8 \pm 334.9}$
Sum	$\underline{1465.4}$	134.0	1084.9	**1612.2**

Table 6 reports MuJoCo return at the recorded 1M evaluation. Table 7 reports time-averaged manipulation return over 50k–500k steps. Table 8 reports the last-100k mean, return AUC, and return at 950k for the MuJoCo benchmark. Table 9 reports return at 500k, return AUC over 50k–500k steps, and sampled ever-success for the manipulation benchmark.

Table 8: Sensitivity analysis: mean ± 95% confidence-interval half-width across three training seeds per cell. Return AUC is the time-averaged return over 25k–1M steps.

Environment	SAC	SAC-Flow	DIME	QSM	DEFlow
Last 100k mean					
Hopper	2775 ± 644	3328 ± 591	2416 ± 627	2491 ± 1277	3165 ± 1126
Walker2d	4487 ± 1604	4443 ± 844	4849 ± 964	1461 ± 2336	5169 ± 522
HalfCheetah	14098 ± 2417	11533 ± 3924	13501 ± 1278	5982 ± 999	12723 ± 449
Ant	5737 ± 859	6286 ± 530	6047 ± 1171	-25 ± 6	5824 ± 2296
Humanoid	4345 ± 1267	5023 ± 107	4875 ± 512	4539 ± 1046	4357 ± 1914
Return AUC					
Hopper	2157 ± 1123	2644 ± 583	2500 ± 164	1956 ± 762	2858 ± 567
Walker2d	2873 ± 2416	3112 ± 176	3127 ± 1230	879 ± 1219	3583 ± 628
HalfCheetah	11220 ± 999	9806 ± 2582	11564 ± 461	5237 ± 610	10465 ± 1491
Ant	3569 ± 715	4917 ± 846	4470 ± 1235	-27 ± 4	3846 ± 651
Humanoid	2902 ± 462	3400 ± 282	3150 ± 816	2722 ± 559	2706 ± 496
Return at 950k					
Hopper	3543 ± 828	3559 ± 45	2251 ± 2148	2996 ± 1308	2725 ± 1920
Walker2d	4423 ± 1504	3748 ± 2373	4757 ± 1307	1035 ± 1551	5131 ± 512
HalfCheetah	13709 ± 2787	11624 ± 3846	13071 ± 3973	6196 ± 917	12677 ± 560
Ant	5682 ± 973	6145 ± 1255	6046 ± 1664	-25 ± 2	5583 ± 2774
Humanoid	4441 ± 1126	4848 ± 743	5220 ± 459	4407 ± 2007	4250 ± 3408

Table 9: Manipulation benchmark results, mean ± 95% confidence-interval half-width across three training seeds per cell. Return AUC is the time-averaged return over 50k–500k steps.

Environment	SAC	SAC-Flow	DIME	DEFlow
Return at 500k				
PickCube	30.2 ± 28.3	23.2 ± 6.6	38.7 ± 7.1	35.5 ± 3.7
PushCube	40.3 ± 11.2	39.8 ± 10.5	42.7 ± 1.0	42.8 ± 1.2
ButtonPressWall	845.6 ± 108.9	35.9 ± 89.1	772.8 ± 155.2	747.6 ± 200.3
PegInsertSide	1274.5 ± 308.7	2.0 ± 0.5	699.7 ± 1607.9	1091.7 ± 693.2
PushWall	78.7 ± 118.1	7.7 ± 19.0	15.8 ± 12.5	457.8 ± 1887.2
Return AUC				
PickCube	25.1 ± 14.6	19.0 ± 3.7	29.1 ± 7.5	33.5 ± 1.2
PushCube	38.4 ± 1.8	37.3 ± 3.1	37.3 ± 3.7	40.8 ± 0.6
ButtonPressWall	641.6 ± 226.3	47.2 ± 138.2	696.7 ± 74.8	755.7 ± 61.8
PegInsertSide	699.1 ± 519.5	2.5 ± 0.1	298.2 ± 589.7	564.3 ± 394.2
PushWall	61.2 ± 97.6	28.1 ± 95.6	23.4 ± 48.3	217.8 ± 831.9
Sampled ever-success (%)				
PickCube	64.7 ± 139.1	5.0 ± 21.5	93.0 ± 23.7	87.3 ± 20.7
PushCube	97.0 ± 9.0	97.3 ± 7.6	99.0 ± 4.3	98.7 ± 2.9
ButtonPressWall	0.0 ± 0.0	0.0 ± 0.0	3.7 ± 5.2	8.7 ± 35.2
PegInsertSide	72.3 ± 24.4	0.0 ± 0.0	23.7 ± 101.8	39.3 ± 53.3
PushWall	0.0 ± 0.0	10.3 ± 44.5	1.0 ± 4.3	0.3 ± 1.4

C.3 Ablation Studies

Integrator step-count ablation. Both figures use the wide-goal, tight-docking MultiGoal variant and two frozen policies, each keeping its training integrator (explicit Euler, implicit DEFlow), both trained at $T = 4$ (seed 3, 200005 transitions). We vary only the docking step count $T \in \{2, 4, 8, 16\}$, keeping navigation and all inputs fixed; this is the only seed with both final checkpoints, since explicit seeds 1 and 2 failed numerically. Figure 8 evaluates both policies on the same 104 reached states with 16 Gaussian inputs each (1664 actions), using balanced Hit@0.06, the per-goal hit rates averaged with equal weights. Figure 9 follows 1000 new stochastic $T = 4$ episodes per policy and recomputes only the docking action from each recorded state, Gaussian base sample, and full batch; replaying at $T = 4$ reproduces the recorded actions bit-for-bit and matches every recorded episode success. Its Hit is the fraction of the 1000 docking actions within 0.06 of the reached goal's target, so frequent routes dominate.

On the shared states, DEFlow stays high for $T \geq 4$ (94.1%, 96.5%, 95.1%; 54.9% at $T = 2$), while the explicit policy reaches 50.1% at its training depth, accurate on only two goals and near zero on the other two, and degrades at smaller and larger step counts (1.4%, 35.1%, 15.1% at $T = 2, 8, 16$). On its own trajectories, explicit succeeds on 99.8% of episodes at $T = 4$ because they visit almost only those two goals, then drops to 0.7% at $T = 2$ and 59.1%/28.2% at $T = 8/16$; DEFlow holds 96.5%, 99.1%, 98.0%, and 53.7% at $T = 2$. In this pair, DEFlow's docking is nearly insensitive

to the tested step counts, whereas the explicit policy's accuracy is route-selective and strongly step-dependent (paired illustration, one seed, each policy with its own integrator).

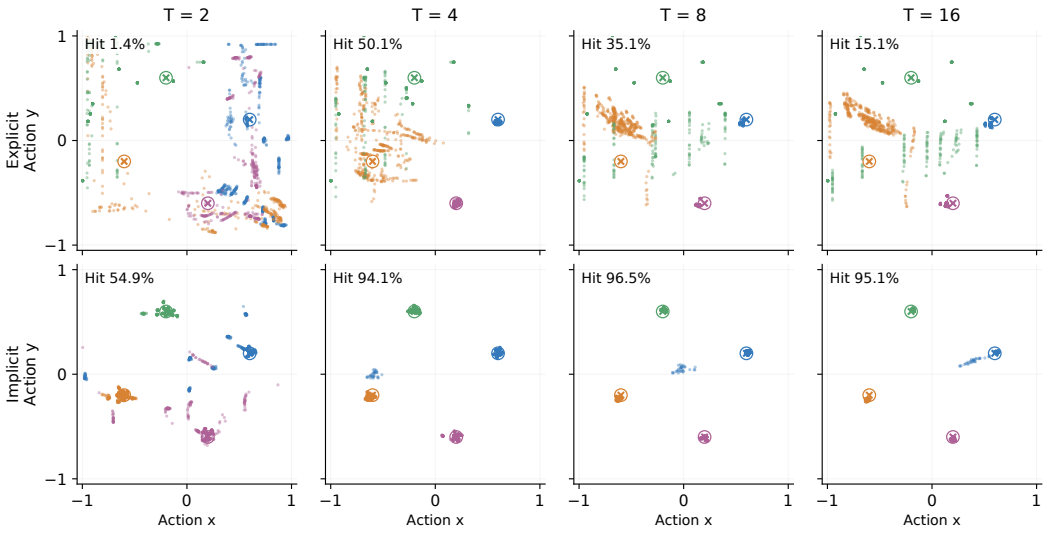


Figure 8: Docking actions on shared states as T varies. Top: explicit; bottom: DEFlow. Hit averages equally over goals.

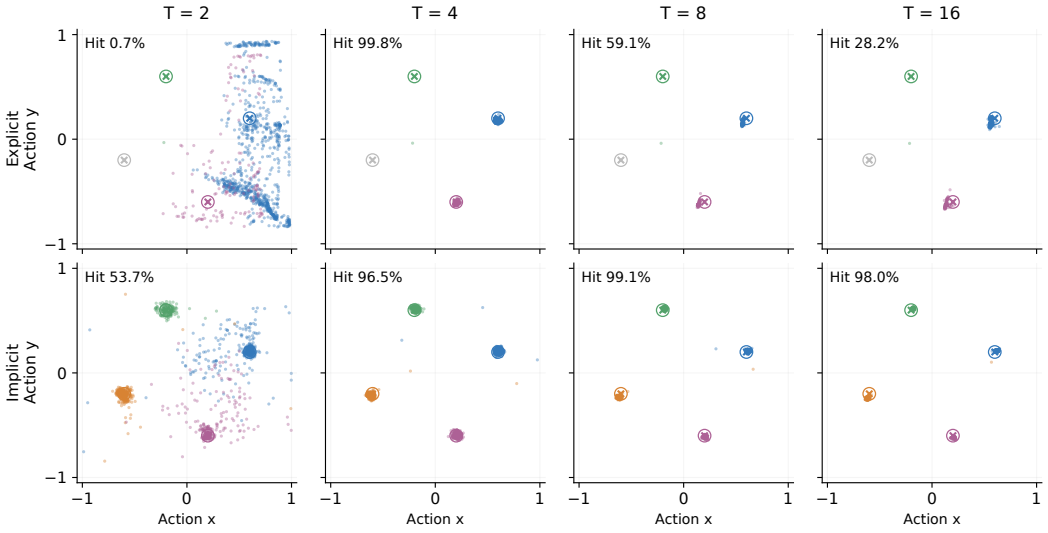


Figure 9: Docking actions on each policy's own trajectories. Hit uses observed goal frequencies; gray targets were not visited.

C.4 Docking Failure Diagnostics

Explicit flow. The docking audit illustrates the step-map conditioning effect identified in Sec. 3. In the notation of Eq. (4), the explicit-step Jacobian is $B_k = D_a F_k^{\exp} = I + \Delta t J_k$. Three original explicit-flow runs terminated after detecting non-finite TD targets. At a frozen checkpoint from one of these runs, $\det(B_0 + \varepsilon I)$ changes sign between two sampled latent inputs. A float64 probe locates a singularity of this diagonally regularized matrix between them. At the same local J_0, the algebraic implicit counterpart $I - \Delta t J_0$ has minimum singular value 1.226. The checkpoint thus exhibits the singular explicit-step regime illustrated by the linear contraction in Appendix A.1.

QSM. QSM's zero docking success is not a failure to reach the goal: 1,374 of 3,000 episodes across three seeds reached the docking state, yet none docked. The failure lies in action generation. At a frozen checkpoint, ideal goal-center observations produced action-box corners in 2,045 of 2,048 samples. An exact replay of the five denoising steps shows that the final learned correction has median magnitude 16.0–21.6 in a box of $[-1, 1]^2$, pushing both coordinates outside the box in 99.4–100% of samples, so clipping maps the action to a corner. All values are finite and the final step adds no fresh noise. Why the correction is learned so large remains unresolved.

D AI Assistance

We used AI tools to assist with writing and polishing the manuscript, to support literature retrieval and discovery of related work, and to draft sections of the paper. The authors reviewed and revised the resulting text and take full responsibility for the final manuscript.